

FASE 1

Nama: Raul Sebastian Sitorus

Kelompok : Copppper

1. Problem 1 : The Last Astronout (Pseudocode)

ALGORITMA The_Last_Astronaut

INPUT N, K

SET status astronot 1..N = AKTIF

SET pos = 1

SET sisa_astronot = N

WHILE sisa_astronot > 1 DO

 hitung = 0

 // 1. Cari astronot ke-K yang masih AKTIF

 WHILE hitung < K DO

 IF astronot[pos] == AKTIF THEN

 hitung = hitung + 1

 ENDIF

 IF hitung < K THEN

 pos = (pos MOD N) + 1

 ENDIF

 ENDWHILE

 // 2. Eliminasi astronot di posisi saat ini

 astronot[pos] = ELIMINASI

 sisa_astronot = sisa_astronot - 1

 CETAK "Eliminasi: Astronot " + pos

 // 3. Perbarui nilai K berdasarkan nomor astronot (pos)

 IF pos GENAP THEN

 K = K + 2

```

ELSE
    K = K - 1
ENDIF

IF K < 2 THEN
    K = 2
ENDIF

// 4. Geser ke posisi berikutnya untuk putaran selanjutnya
pos = (pos MOD N) + 1
ENDWHILE

CETAK "Astronot terakhir yang bertahan: " + astronot yang masih AKTIF

```

Penjelasan Logika :

Logika utama dari program ini meniru sebuah permainan eliminasi lingkaran, mirip dengan permainan anak-anak saat menentukan giliran secara berputar. Bayangkan ada sekelompok astronot yang berdiri melingkar dan diberi nomor urut dari 1 sampai N. Program akan mulai menghitung dari astronot pertama sebanyak **K langkah**. Setiap kali hitungan berjalan, program akan mengecek status astronot tersebut. Jika astronot masih aktif (belum tereliminasi), hitungan akan bertambah 1. Proses melingkar ini memanfaatkan fungsi sisa bagi (MOD), sehingga saat hitungan mencapai astronot paling akhir, posisi hitungan otomatis kembali lagi ke astronot pertama di awal lingkaran tanpa terputus. Begitu hitungan tepat mencapai nilai K, astronot di posisi tersebut akan langsung dieliminasi dan statusnya diubah menjadi true.

Permainan ini menjadi unik karena memiliki **aturan dinamis** yang membuat nilai K (jumlah langkah hitungan) selalu berubah setiap kali ada astronot yang gugur. Jika astronot yang tereliminasi memiliki nomor punggung genap, maka tantangan diperberat dengan menambah nilai K sebanyak 2 langkah untuk putaran berikutnya. Sebaliknya, jika nomornya ganjil, nilai K dikurangi 1 langkah, dengan batas minimal K tidak boleh kurang dari 2 agar permainan tetap berjalan adil. Setelah menentukan nilai K yang baru, program akan mencari astronot hidup terdekat yang berada setelah posisi eliminasi tadi untuk dijadikan titik awal hitungan berikutnya. Proses hitung, eliminasi, dan perubahan aturan K ini akan terus diulang di dalam lingkaran sampai seluruh astronot gugur, sehingga program dapat mencatat siapa astronot paling terakhir yang berhasil bertahan.

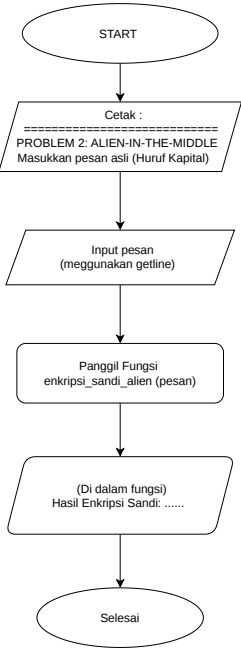
Problem 2 : Alien-In-The-Middle (Flowchart)

Penjelasan logika :

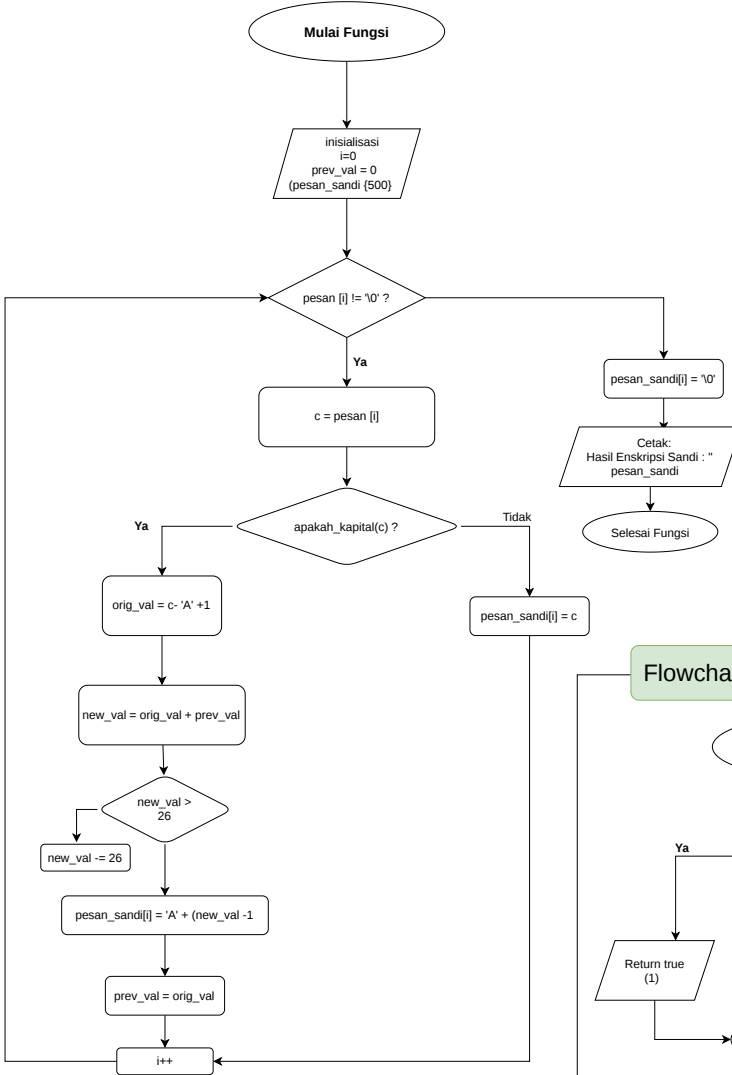
Fungsi Utama dimulai dengan menampilkan teks instruksi lalu menerima masukan kalimat dari pengguna menggunakan `getline`. Setelah itu, program memanggil Fungsi Enkripsi (`enkripsi_sandi_alien`) untuk memproses pesan tersebut. Sebelum karakter diproses, fungsi pembantu `apakah_kapital` akan memeriksa apakah suatu karakter berada di rentang 'A' sampai 'Z'. Jika benar, fungsi pembantu mengembalikan nilai `true` (1), dan jika bukan (seperti spasi atau tanda baca), mengembalikan `false` (0).

Di dalam Fungsi Enkripsi, program melakukan perulangan (*looping*) pada tiap karakter hingga menemukan karakter pengakhir *string* (`"\0"`). Jika karakter bukan huruf kapital, nilainya langsung disalin ke `pesan_sandi`. Namun jika karakter adalah huruf kapital, nilainya diubah menjadi urutan alfabet (A=1, B=2, dst.), lalu dijumlahkan dengan `prev_val` (nilai urutan huruf kapital asli sebelumnya). Jika hasil penjumlahan `new_val` melebihi 26, nilainya diisi kembali dengan 26 (posisi 'Z'). Karakter sandi baru kemudian dihitung kembali ke bentuk karakter (`'A' + new_val - 1`), nilai `prev_val` diperbarui dengan nilai asli huruf tersebut, dan indeks `i` bertambah untuk memproses karakter berikutnya hingga pesan selesai dienkripsi dan dicetak. Bentuk flowchartnya dapat dilihat pada halaman selanjutnya.

Flowchart Fungsi Utama



Flowchart fungsi enkripsi



```

graph TD
    Start([Mulai fungsi]) --> Decision{c >= 'A' <br/> dan <br/> c <= 'Z' ?}
    Decision -- Ya --> True[/Return true<br/>(1)/]
    Decision -- Tidak --> False[/Return False<br/>(0)/]
    True --> End([Selesai Fungsi])
    False --> End
  
```

